

vector 정리

크기가 자동으로 늘어나는 배열입니다. C++ STL에서 가장 자주 쓰는 기본 컨테이너입니다.

사용처 (Where)

- 입력 개수가 미리 정해지지 않았을 때
- 인덱스로 빠르게 접근해야 할 때
- 정렬, 탐색 알고리즘과 함께 쓸 때

방법 (How to Use)

- 헤더 포함 후 `vector<자료형>` 이름 형태로 선언
- `push_back`으로 뒤에 추가
- 인덱스, `at()`, 반복문으로 접근
- `sort(v.begin(), v.end())`처럼 알고리즘과 결합

기본 정보

- 필요 헤더: `#include <vector>`
- 기본 선언: `vector<int> v; vector<string> names(3);`

왜 써야 할까? (Why)

- 배열보다 크기 관리가 쉽고 안전함
- 대부분의 STL 알고리즘과 바로 연결됨
- 자료 개수가 변하는 문제에서 코드가 짧아짐

주요 함수

함수/문법	의미
<code>push_back(x)</code>	맨 뒤에 x 추가
<code>pop_back()</code>	맨 뒤 원소 제거
<code>size()</code>	원소 개수
<code>empty()</code>	비었는지 확인
<code>at(i)</code>	범위 검사하며 i번째 접근
<code>front()/back()</code>	첫/마지막 원소

기본 코드 예시

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    cout << "size: " << v.size() << '\n';
    cout << "first: " << v[0] << '\n';

    for (int x : v) {
        cout << x << ' ';
    }
}
```

vector 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
v[i]	O(1)
push_back	평균 O(1)
insert/erase 중간	O(n)
sort	O(n log n)

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 점수 목록 정렬 후 상위 점수 출력

입력받은 점수를 vector에 저장하고 내림차순 정렬하여 상위 3개를 출력합니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> score = {70, 95, 82, 100, 88};
    sort(score.begin(), score.end(), greater<int>());

    for (int i = 0; i < 3 && i < score.size(); i++) {
        cout << score[i] << ' ';
    }
}
```

처음 배울 때 자주 하는 실수

- 빈 vector에서 v[0], front(), back()을 호출하지 않기
- 반복 중 erase를 쓰면 iterator가 무효화될 수 있음
- 중간 삽입/삭제가 많으면 list/deque도 검토

비슷한 항목과 차이

- array는 크기가 고정이고 vector는 동적 크기
- list는 중간 삽입/삭제에 유리하지만 인덱스 접근이 느림
- deque는 앞뒤 삽입/삭제가 모두 빠름

한 줄 정리: vector은(는) '입력 개수가 미리 정해지지 않았을 때' 상황에서 먼저 떠올리면 좋은 STL입니다.